LANGUAGE IS A DIRECTION IN EMBEDDING (LAYER 0) SPACE

Plinio Avendaño
Department of Computer Science
UTEC
plinio.avendano@utec.edu.pe

ABSTRACT

The linear representation hypothesis (LRH) states that concepts are encoded as linear subspaces of a model's representation space. This means that there exist directions that represent concepts such as gender, sentiment, refusal, etc. This was first observed in word embeddings where simple arithmetic revealed a direction for gender that could be added to another embedding to steer its meaning (the famous $king - man + woman = queen$). In LLMs, the LRH has been studied for the model's internal and output representations and has motivated lines of research such as superposition and activation steering. However, almost all of this work operates on hidden layers, and whether the input embedding space of an LLM still contains directions that the model reads as properties of its input remains largely unexplored. In this work, using Llama-3.2-3B-Instruct, we find (through gradient-based optimization) a single vector that, when added identically to every token embedding of the user's question, makes the model answer in French in 100% of held-out English questions with 96% accuracy, while every token remains closest to its original embedding. The effect transfers to input languages unseen during training (Italian, Portuguese), and the same holds for Spanish and German directions. Moreover, the effect is signed and language-specific. Subtracting the French vector from French questions reduces French responses from 100% to 12%, whereas subtracting the Spanish or German vector achieves a merely 4–10%. Behavioral tests suggest the vector alters the model's perception of the input rather than acting as an instruction. When asked to identify the language of a patched English question, the model answers "French" in 76% of cases (0% for a random vector of the same norm), and an explicit textual instruction overrides it. These results show that input language is a direction and linearly accessible at the embedding layer (before any computation by the network). This brings the LRH back to the embedding space where it began.

1 Introduction

The linear representation hypothesis (LRH) states that high-level concepts are represented as linear directions, or more generally linear subspaces, in a model's representation space (Park et al., 2024). It started as an observation from Mikolov et al. (2013), who found that word embeddings encode semantic and syntactic relationships that simple arithmetic can recover ($king - man + woman \approx queen$). This is interesting because it implies that $king - queen \approx man - woman$; that is, there is a direction for gender. Remarkably, this happens without being the explicit objective of the algorithm, learning a dimensional mapping from words to vectors yields this kind of emergent relationship.

In LLMs the hypothesis has been extended to the hidden states of the model (activations/attention heads), and it has shaped the field in two ways. First, by opening new lines of research, and second, by enabling practical methods to control model behavior.

Works such as Toy Models of Superposition by Elhage et al. (2022), where they find that networks represent more concepts than they have neurons by storing them in 'superposition', build on this. Sparse autoencoders (Bricken et al., 2023; Templeton et al., 2024) take this one step further, decomposing these superposed activations into interpretable features. On the other hand, activation steering is a method to control model behavior that relies on linear directions. It works by subtracting the activations of contrastive pairs of prompts in middle to top layers, which yields directions that can enhance or suppress behaviors. Using this method, Arditi et al. (2024) showed that refusal is mediated by a single direction in the residual stream, and Ball et al. (2024) showed that different jailbreaks also share a common direction.

However, almost all of this work, both on reading features and on writing them, operates on the hidden layers of the model. The input embedding space, where the LRH was born, has received much less attention in LLMs: Dar et al. (2023) use it to interpret the parameters of the model, and Freenor and Alvarez (2025) argue that modern text representations have lost the simple vector arithmetic of early word embeddings and in consequence the authors model semantic transformations as rotations in the output space of embedding models. Whether the input embedding space of an LLM still contains linear directions that the model reads as properties of its input remains largely unexplored. We intervene in this space, hypothesizing that if adding a single vector to every token changes what the model perceives about its input, that property is linearly accessible at layer 0.

In this work, we show that language is a direction that, when added to the token embeddings of the user's prompt, changes the language of the model's answer to French, both from English and from other input languages. This is achieved by using gradient descent to optimize a single vector, with the objective of minimizing the cross-entropy of French target answers under teacher forcing, over a collection of questions in English, Spanish, and German. Moreover, applying the same procedure, we also find analogous directions for Spanish and German. Our contributions are as follows:

- We show that the input embedding space of an LLM contains linear directions that the model reads as the language of its input. In Llama-3.2-3B-Instruct and Qwen3-4B-Instruct-2507, adding a single vector to every token of an English question makes the model answer in French in 100% and 97% of held-out questions (respectively) while every token remains closest to its original embedding. Even more, the model retains the same accuracy as when the question is written in French.
- We show that the effect reverses with the sign of the vector and is largely language-specific. Subtracting a language's vector from questions written in that language moves the answer out of it. In Llama, only 3–17% of the answers remain in the language of the question. In Qwen, this holds for German (7%), while for French and Spanish the effect is partial (52–62% remain). Subtracting another language's vector leaves at least 80% of the answers in the original language, except between French and German in Llama, and random vectors of the same norm leave 92–100%. In contrast, position-bound patches trained on the same task cannot be inverted.
- We introduce behavioral tests that distinguish a change in the perceived input from an instruction, and show that the direction acts on perception. Asked which language a patched English question is written in, the model names the language of the patch in 76–90% of cases in Llama and 88–99% in Qwen (0% for a random vector).
- We discuss the limits of this view, the directions are not unique across training runs, share a common component across languages, and do not compose additively at the input. (COMPLEMENTAR)

2 Background

We now introduce the definitions necessary for this work.

2.1 Transformer notation

We consider a decoder-only transformer with L layers and hidden dimension d. Given an input sequence of tokens $x_1, \ldots, x_T$ from a vocabulary $\mathcal{V}$, each token is mapped to a vector by the embedding matrix $W_E \in \mathbb{R}^{|\mathcal{V}| \times d}$:

$$e_i = W_E[x_i] \in \mathbb{R}^d. \tag{1}$$

These token embeddings form the initial state of the residual stream, which we refer to as layer 0:

$$h_i^0 = e_i. \tag{2}$$

Each layer ℓ then reads from the residual stream and adds its output back to it. In both models we study, Llama-3.2 (Touvron et al. 2023 and Grattafiori et al. 2024) and Qwen3 (Yang et al. 2025), each layer consists of an attention block and an MLP block, both preceded by an RMSNorm (Zhang & Sennrich 2019):

$$\tilde{h}_i^\ell = h_i^\ell + \mathrm{Attn}^\ell\big(\mathrm{RMSNorm}(h^\ell)\big)_i, \tag{3}$$

$$h_i^{\ell+1} = \tilde{h}_i^\ell + \mathrm{MLP}^\ell\big(\mathrm{RMSNorm}(\tilde{h}_i^\ell)\big), \tag{4}$$

where $\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}} \odot g$, with a learned gain $g \in \mathbb{R}^d$. The attention block is the only component that moves information between positions: position i can only attend to positions $j \le i$. After the last layer, the next-token logits are obtained as

$$\mathrm{logits}_i = W_U\, \mathrm{RMSNorm}(h_i^L), \tag{5}$$

where $W_U \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the unembedding matrix. For the two models, the embedding and unembedding matrices are tied, that is, $W_U = W_E$.

Two properties of this architecture are relevant to our work. First, since every layer adds to the residual stream rather than overwriting it, a vector added to h_i^0 remains present in all subsequent layers, unless later layers write in the opposite direction. Second, every block reads the residual stream through an RMSNorm, so what a layer sees depends on the direction of h_i^ℓ relative to its norm, not only on its absolute value.

2.2 Concepts or features

Elhage et al. (2022) provide three working definitions for features:

- Features as arbitrary functions: Any function of the input.
- Features as interpretable properties: "The presence of human understandable 'concepts' in the input". Such as gender, sentiment, etc.
- Neurons in sufficiently large models: Properties of the input to which a sufficiently large model would dedicate a neuron.

We define language (such as French) as a concept, so naturally we will stick with definition number two and will use concept and feature as analogous words. Nevertheless, it is important to note that the authors also acknowledge that this definition may not be enough, because models may represent features that we humans do not understand. However, because in this work we are dealing with human concepts this definition suffices.

They also hypothesize that features are represented as directions in *activation space*, even though the features themselves are nonlinear functions of the input. That is, a feature such as the language of a text is computed by the network from its tokens, and only after passing through sufficient layers the concept is represented linearly. This makes the input embeddings, where no such computation has happened yet, an unlikely place for it.

Yet their main example of features as directions comes from embeddings, the gender direction of Mikolov et al. (2013). Therefore, the input embeddings of an LLM are also a natural place to look for this kind of structures. In a transformer, the embedding matrix is not a fixed lookup table, it is learned jointly with the rest of the model during pretraining (Vaswani et al. 2017), so it is shaped by the same training pressure as the hidden layers. For this reason, we hypothesize that it may already contain directions that the model can read as concepts. This is the space we study in this work.

2.3 Soft prompts

Adapting a pretrained model to a new task usually requires either fine-tuning its weights or writing a text prompt that describes the task. Soft prompts offer a middle ground, instead of words, the prompt is made of continuous vectors that are learned directly from data, while the model's weights stay frozen.

Prompt tuning (Lester et al. 2021) implements this by prepending k trainable vectors $p_1, \ldots, p_k \in \mathbb{R}^d$ to the token embeddings of the input:

$$[\, p_1, \ldots, p_k, e_1, \ldots, e_T \,]. \tag{6}$$

These vectors live in the same space as the token embeddings, but they do not need to correspond to any token in the vocabulary. They are optimized by gradient descent on the task loss, so that the frozen model, reading them as if they were part of the prompt, performs the desired task. Beyond task adaptation, the same idea has been used to find universal jailbreaks (Schwinn et al. 2024) and to compress long texts into a single input vector (Kuratov et al. 2025).

Soft prompts are relevant to this work because they also optimize vectors in the embedding space of a frozen model. The crucial difference is that soft prompts are bound to the positions where they were optimized. Nothing in their training forces them to represent a direction or a transformation of the prompt; rather, they behave as a compressed instruction (Mu et al. 2023). The same holds when the vectors are added to the prompt tokens instead of concatenated (like in Schwinn et al. 2023 set up), because each vector is optimized for a specific token it does not carry over to other tokens (appendix 2). In contrast, the direction we study adds no positions and does not depend on the input. A single vector is added identically to the embedding of every token of every question.

This difference also shows in how each can be manipulated. Arithmetic on soft prompts has been explored, but as interpolation between the parameters of trained prompts, used to combine classification tasks or to initialize new ones (Belanec et al. 2024); it is not the representation-level arithmetic of word embeddings (Mikolov et al. 2013) or function vectors (Todd et al. 2024), and inverting a behavior by negating a soft prompt has not been shown. In Results, we observe that our direction behaves differently from a position-bound patch.

3 Method

3.1 The patch (direction)

We use Llama-3.2-3B-Instruct ($L = 28$, $d = 3072$, $|\mathcal{V}| = 128{,}256$) and Qwen3-4B-Instruct-2507 ($L = 36$, $d = 2560$, $|\mathcal{V}| = 151{,}936$). Inputs follow each model's chat template, with a system block, a user block containing the question, and the header of the assistant turn. Let $\mathcal{Q}$ be the set of positions of the question tokens in the user block. The patch is a single vector $v \in \mathbb{R}^d$ that is added as follows:

$$e'_i = \begin{cases} e_i + \alpha v & \text{if } i \in \mathcal{Q}, \\ e_i & \text{otherwise,} \end{cases} \tag{7}$$

where α is a scalar coefficient. We use $\alpha = 1$ to add the direction and $\alpha = -1$ to subtract it. The system prompt, the role headers, and special tokens such as `<|eot_id|>` in Llama or `<|im_end|>` in Qwen are left untouched, and so are the tokens the model generates. The patch modifies only the user section of the prompt.

Because the same vector is added at every position of the question, it cannot depend on where a token appears in the sequence or on which token it is added to. This is what distinguishes it from a soft prompt, and it makes v a candidate direction of the embedding space rather than a set of learned tokens. Whether the model reads it as a direction is what the tests of Behavioral Results evaluate.

To compare our method with soft prompting, we also train two patches with position-bound formats and the same objective. This gives three formats:

- **Shared (ours).** A single vector v, added to every question position as in Eq. (7).
- **Question-3.** Three distinct vectors, added to the first three tokens of the question, similar to Schwinn et al. (2023) but shared across inputs.
- **Header-3.** Three distinct vectors, added to the three tokens of the assistant header (`assistant`, `<|end_header_id|>`, `\n\n` in Llama, and `<|im_start|>`, `assistant`, `\n` in Qwen). Since these positions are the same for every input, this is equivalent to a soft prompt (Lester et al. 2021).

3.2 Finding the patch (training)

The patch is optimized directly, with the model frozen. For a target language t, we build a training set of pairs (q, y), where q is a question written in a source language $s \neq t$ and y is an answer to that question written in t. Questions in the target language are excluded from training. For French, the source languages are English, Spanish, and German.

Given a pair (q, y), we patch the embeddings of q as in Eq. (7) and compute the cross-entropy of the target answer under teacher forcing, averaged and restricted over its first K tokens:

$$\mathcal{L}(v) = \mathbb{E}_{(q,y)} \left[-\frac{1}{K} \sum_{k=1}^{K} \log p\big(y_k \mid q \oplus v,\ y_{<k}\big) \right] + \lambda \|v\|_2^2, \tag{8}$$

where $q \oplus v$ denotes the prompt with the patch applied to the question tokens, and λ controls an L_2 penalty on the norm of the patch. We use $K = 8$, the first tokens of the answer are where the model commits to a language, and once it does, it maintains it for the rest of the generation. Restricting the loss to them makes the objective about the language of the answer rather than about reproducing a particular answer or style.

We optimize v with sign-SGD, which updates each coordinate by a fixed step in the direction of the sign of its gradient:

$$v \leftarrow v - \eta_t \operatorname{sign}\big(\nabla_v \mathcal{L}\big), \qquad \eta_t = \frac{\eta_0}{2}\left(1 + \cos\frac{\pi t}{T-1}\right), \tag{9}$$

with $\eta_0 = 2.5 \times 10^{-4}$ and a cosine schedule that anneals the step to zero over the T update steps.

Each update uses the average gradient over a batch of 28 training questions, each presented in its three source languages, accumulated with one forward and backward pass per pair. We take 20 update steps on each batch before moving to the next one. We train for 10 epochs of 21 batches, for a total of $T = 4{,}200$ updates, with $\lambda = 0.0915$. The patch is initialized at zero and the order of the batches is fixed, so training is deterministic.

The patches for Spanish and German are trained with the same recipe, using the other two languages as sources.

3.3 Data

Questions. We use a bank of 700 short questions with a verifiable answer, divided into seven categories of 100 questions that differ in the form of the prompt:

- *Factual*: *What is the capital of Iceland?*
- *Factual with other openings*: *Where was Mozart born?*
- *Imperative*: *State the capital of Mongolia.*
- *Explanatory imperative*: *Detail why ice floats on water.*
- *Explanatory question*: *How do geckos climb walls?*
- *Short fragment*: *Capital of Peru?*
- *Conversational*: *Hey, who directed Jaws?*

All questions are at most 10 tokens long. Both the questions and their translations were generated by Claude Opus 5. Each question is translated into French, Spanish, and German, which are the target languages of the patches, and into Italian and Portuguese, which are used only for evaluation and never appear during training. We reviewed every translation by hand against the English original, checking that it preserves both the meaning and the form of the question (an imperative stays an imperative, a fragment stays a fragment, and a conversational question keeps its casual tone). This review corrected 38 translations in 31 questions (mostly mistranslated titles, changes in meaning/form, and grammar errors). The bank is interleaved by category, so splitting it by position keeps it balanced. The first 588 questions are used for training and the last 112 (16 per category) are held out for evaluation. We select questions with a verifiable answer so that we can measure whether the patch induces errors or hallucinations, and we vary their form so that the patch cannot rely on a single question template.

Target answers. The target y for each question is the model's own answer to the question written in the target language, without any instruction:

$$y = M\big(q_{\text{lan}}\big), \tag{10}$$

where lan is the language of the question. Targets are generated with greedy decoding for up to 250 tokens, so that explanatory answers are not cut before the explanation. Each model is trained on its own answers, so Llama-3.2-3B-Instruct and Qwen3-4B-Instruct-2507 generate their targets separately from the same questions. Since the target is what the model answers when the question is genuinely written in the target language, and not what it answers when told to respond in that language, the patch is trained to reproduce an input in that language rather than an instruction.

We reviewed all 2,100 targets of each model by hand. We kept the style of the model and corrected factual errors (e.g., *Jean-Pierre Jeunet* as the director of *Jaws*), words in another language, and filler openings such as *¡Claro!*. This changed 447–488 of the 700 targets per language in Llama and 526–557 in Qwen. It is important to note that most of these changes correct factual errors (350–410 targets in Llama and 414–482 in Qwen), but only a minority of them concern the answer itself: 124–138 targets in Llama and 70–96 in Qwen did not contain the correct answer, while the rest gave the correct answer followed by errors in the explanation (wrong dates, wrong mechanisms, or invented names). We also made sure that no target is a bare answer (e.g., *Pretoria* instead of *La capital de Sudáfrica es Pretoria*), since a bare answer is often identical across languages and gives the patch no signal about the language. After the review, every target is in the target language and answers correctly. These corrected targets are used only for training. In evaluation, we compare the patch against the model's answers to the questions written in the target language. These answers are generated again at evaluation time and are not corrected.

Open-ended prompts. To test generalization beyond the question formats seen in training, we use two additional sets that never enter training: 99 open-ended prompts (*What's the weather like…*) and 50 imperative prompts (e.g., *Tell me about…*, *Explain…*), which differ from the training questions in both form and length. No imperative in the training bank starts with a verb used in the second set, so it still measures transfer to unseen imperative verbs. Both sets are also translated into French, Spanish, and German, which provides the native control on them.

3.4 Evaluation

We report two main metrics:

- **Language rate:** The fraction of responses written in the target language, as identified by the language detector (GlotLID) on the full response.

- **Accuracy:** The fraction of responses that answer the question correctly, measured by matching the response against a list of accepted answers for each question, with aliases in every evaluation language (e.g., *Athens*, *Athènes*, *Atenas*, *Athen*).

The language rate measures whether the patch changes the language of the answer, and accuracy measures whether it does so without damaging the model's reading of the question. We compare three conditions on the held-out questions. *None* is the English question without the patch. *Native* is the question written in the target language, without the patch, and it serves as the reference for how often and how accurately the model answers in that language. Its answers are generated at evaluation time and are not corrected. *Patch* is the English question with the patch added.

We report these metrics with the held-out dataset as well as the open sets (without accuracy due to being open).

3.5 Behavioral tests and controls

Beyond showing that the patch switches the language of the answer, we test two properties. First, whether its effect reverses with its sign (as expected from a direction). Second, whether it acts on the model's reading of the input or on its decision about the output. All tests use the held-out questions on the three patches.

Subtraction. For each language lan $\in \{\text{fr}, \text{es}, \text{de}\}$, we subtract the patch v_{lan} from questions written in lan ($\alpha = -1$ in Eq. (7)), which the model already answers in lan without any patch, and measure the distribution of response languages and accuracy. A direction should move the answer out of lan, while a set of learned tokens has no reason to. We compare with three controls. First, we subtract the patches of the other two languages, which share a common component with v_{lan} but encode a different language. If the effect is specific to lan, these should leave the answer in lan. Second, we subtract random vectors with the same norm as v_{lan}, to test whether any displacement of this size is enough to change the language. Third, we subtract the position-bound Question-3 and Header-3 patches of the same language, each at the positions where it was trained. If these act as learned tokens rather than as a direction, subtracting them should leave the answer in lan.

Input or instruction. A patch that makes English questions receive French answers admits two readings. It could alter the model's reading of the input, making it perceive the question as written in French, so that the model answers in French by following its usual behavior of replying in the language of the question. Alternatively, it could act on the decision about the output language, as the instruction *Answer in French* would. We argue that the first reading is correct, and we refer to it as a "translation" of the input (a change in the perceived language of the question, not in its tokens). Standard metrics cannot distinguish the two readings, so we design three tests in which the patch is applied only to the span of the question, embedded in a longer message that asks for something else:

- **Language identification.** We ask the model which language the question is written in (*What language is the following question written in? Reply with only the name of the language.*) and record the first language name in the response. If the patch alters the reading of the input, the model should report the language of the patch. We run this test with the patches of all three languages, and also with the position-bound Question-3 and Header-3 patches. If these act as an instruction rather than on the input, the model should report English.
- **Subtraction against an instruction.** We prepend *Answer this in French.* to a French question and subtract the French patch from the question tokens. If the patch encodes the output decision, subtracting it should counteract the instruction. If it encodes a property of the input, the instruction should prevail.
- **Conflicting instruction.** We prepend *Answer this in English.* to a patched English question, and compare with the same instruction applied to a question genuinely written in French. If the patch changes the perceived input, the patched question should behave like the genuine French question, whichever way the model resolves the conflict.

In all three tests, the controls are the unpatched question, the question genuinely written in the target language, and a random vector with the same norm as the French patch.

These tests also position the patch with respect to the two families of methods discussed in Background. A soft prompt adds tokens that act as a compressed instruction, and activation steering writes a behavior into the hidden states of the model; in both cases, the intervention tells the model what to do. The patch, in contrast, modifies the initial representation of the question itself, before any computation by the network. If it acts on the model's reading of the input, the model does not receive an order to answer in French. It receives a question that, in its own representation, is written in French, and answers it as it would answer any French question. The instruction reading would make the patch a continuous counterpart of a soft prompt; the input reading would make it a different kind of intervention, one that edits what the model perceives rather than what it is asked to do. The three tests are designed to tell these two cases apart.

4 Results

All results in this section are computed on the 112 held-out questions of each language, except for the open sets.

4.1 A single vector switches the answer language

On English questions, the French patch turns all answers of Llama into French and 97% of those of Qwen, and the Spanish and German patches reach 96–99% in both models (Table 1). These rates match those of the native questions (95–100%), without any instruction being present in the prompt. Accuracy is also preserved. It drops from 97–99% on English questions to 72–84% on native questions, which is the cost of the model answering in another language, and the patch matches this reference. In Qwen, the patch and the native question differ by at most one question. In Llama, the patch is slightly more accurate than the native question in all three languages, by 3 to 11 questions. The two conditions also fail on largely the same questions. They disagree on 12–14 questions in Qwen and 9–25 in Llama, and these disagreements are split in both directions (Won/Lost).

Table 1: Held-out English questions. $\|v\|$: L2 norm of each patch. None, Native, and Patch as defined in Section Evaluation. Δ: accuracy of the patch minus accuracy of the native question, computed from unrounded values. *Won/Lost*: questions that the patch answers correctly and the native question does not, and vice versa.

			Target language (%)			Accuracy (%)				
Model	Patch	$\|v\|$	None	Native	Patch	None	Native	Patch	Δ	Won/Lost
Llama-3.2-3B	French	0.88	0	99	100	97	76	79	$+2.7$	6/3
	Spanish	0.91	0	96	99	97	72	82	$+9.8$	18/7
	German	0.94	0	95	99	97	78	81	$+3.6$	13/9
Qwen3-4B	French	0.92	0	100	97	99	84	84	0.0	7/7
	Spanish	0.96	1	97	96	99	84	84	0.0	6/6
	German	1.00	0	98	97	99	84	83	-0.9	5/6

The same holds for held-out questions written in the other two training languages of each patch, where the patches switch 96–100% of the answers in Llama and 93–99% in Qwen (see Appendix 1).

Furthermore, the vectors have norms between 0.88 and 0.94 in Llama and between 0.92 and 1.00 in Qwen, which is small compared with the token embeddings they are added to. In Llama, each patch is shorter than more than 91% of the tokens in the vocabulary and than more than 97% of the tokens that appear in the questions, whose median norm is 1.16; in Qwen, the same figures are 84% and 89%, with a median norm of 1.12. We also check that the patch does not rewrite the question into other tokens. For every token of the held-out questions, we take its patched embedding $e_i + v$ and look for the nearest token in the vocabulary, both by Euclidean distance and by cosine similarity. With the patch applied as in all our experiments, the nearest token is always the original one, for every patch in both models and for questions written in English or in either of the other two languages. This remains true when the patch is scaled up to four times its norm, even though at that scale it is longer than the typical distance between a token and its nearest neighbor (0.95–0.96 in Llama and 0.93 in Qwen), so the patch does not point toward other tokens of the vocabulary. The model therefore receives the same tokens, shifted along a direction, rather than a different text.

4.2 The patch generalizes to new languages and open sets

We now test whether the patches work on languages that have not been on the training set and on the open ended sets.

4.2.1 Unseen input languages

Italian and Portuguese never appear in training, neither as source nor as target languages. All three patches still switch most of these questions to the target language, in both models. For Italian questions, Llama answers in the target language 89% of the time with the French patch, 91% with the Spanish one, and 97% with the German one; Qwen does so 100%, 83%, and 89% of the time. Portuguese questions switch just as reliably: 96%, 84%, and 89% in Llama, and 97%, 97%, and 99% in Qwen. Nevertheless, the switch has a cost in accuracy. Without a patch, Llama answers 72% of the Italian questions and 71% of the Portuguese ones correctly, and with a patch this drops to between 57% and 65%. In Qwen, accuracy drops from 73% (Italian) and 86% (Portuguese) to between 64% and 79%.

The patch therefore transfers the language to questions in languages it has never seen, but not the accuracy. On English questions, the patch is as accurate as the native question (Table 1); on Italian and Portuguese questions, it is between 4 and 21 points less accurate (Δ in Table 2).

Table 2: Held-out questions written in languages never used in training. Δ: accuracy of the patch minus accuracy of the native question in Table 1. *Won/Lost*: questions that the patch answers correctly and the native question does not, and vice versa.

			Target language (%)		Accuracy (%)			
Model	Patch	Question in	None	Patch	None	Patch	Δ	Won/Lost
Llama-3.2-3B	French	Italian	0	89	72	62	−14.3	9/25
		Portuguese	0	96	71	62	−13.4	5/20
	Spanish	Italian	0	91	72	58	−14.3	9/25
		Portuguese	2	84	71	63	−8.9	9/19
	German	Italian	0	97	72	57	−20.5	3/26
		Portuguese	0	89	71	65	−12.5	5/19
Qwen3-4B	French	Italian	0	100	73	71	−12.5	5/19
		Portuguese	0	97	86	79	−4.5	5/10
	Spanish	Italian	0	83	73	70	−14.3	1/17
		Portuguese	2	97	86	79	−5.4	5/11
	German	Italian	0	89	73	66	−17.9	3/23
		Portuguese	0	99	86	64	−19.6	3/25

4.2.2 Open sets

On the 99 open-ended prompts, the patches yield 94–96% of answers in the target language in Llama and 89–99% in Qwen, against 98–100% for the native prompt (Table 3). The lowest value is the French patch in Qwen, and its failures concentrate on a single form: 10 of its 11 failures are *How do I . . . ?* prompts, answered in English with the model's usual openings (e.g., *That's a great question!*).

Table 3: Open 1, 99 open-ended prompts in English. Without a patch, the model answers in English (0% in the target language in every row). Native: the prompt written in the target language.

		Target language (%)	
Model	Patch	Native	Patch
Llama-3.2-3B	French	99	94
	Spanish	100	96
	German	99	94
Qwen3-4B	French	98	89
	Spanish	99	99
	German	99	98

Imperative prompts, whose verbs never appear in the training imperatives, switch just as reliably (Appendix 2). From English, 98–100% of the answers switch in Llama and 96–100% in Qwen, and from the other training languages the rate is between 92% and 100% in every case.

4.3 The patch behaves as a direction

Now we test if the patches can be subtracted. We expect that if a patch representing language X really is a direction then given a question in language X subtracting the patch yields an answer outside X. Moreover, subtracting the patch of another language should have a much smaller effect, close to that of a random vector of the same norm. In contrast, the Question-3 and Header-3 patches (section 3.1), should have no inverse, since their vectors are bound to specific positions rather than forming a direction.

Every language patch has an inverse, all in different degrees. German is the strongest in both models, where only 3% (Llama) and 7% (Qwen) of the answers stay in German. French and Spanish keep 9% and 17% of the answers in Llama,

but 62% and 52% in Qwen. In Llama, the answers that leave French and German go mostly to English, while those that leave Spanish go to French. In Qwen, the answers that leave French and Spanish go mostly to Romanian, while those that leave German go to English. Subtracting a random vector of the same norm barely changes the language, with 94–99% of the answers staying.

Table 4: Subtracting each patch from questions written in its own language. Stays: percentage of answers that remain in the language of the question. $-v$: the patch subtracted; $-r$: a random vector of the same norm subtracted, averaged over three vectors. Destination: languages of the answers that leave with $-v$.

		Stays (%)				Accuracy (%)		
Model	Question in	None	$-v$	$-r$	Destination (%)	None	$-v$	$-r$
Llama-3.2-3B	French	100	9	99	English 84, Spanish 6, German 1	76	30	60
	Spanish	99	17	97	French 77, English 3, Italian 1, Portuguese 1	72	46	61
	German	99	3	94	English 96, Dutch 2	78	29	57
Qwen3-4B	French	100	62	97	Romanian 24, Italian 5, Spanish 4, German 3	84	76	77
	Spanish	97	52	95	Romanian 31, Portuguese 12, Catalan 3, English 2	84	45	76
	German	98	7	96	English 63, Dutch 10, Romanian 7, Spanish 6	84	27	71

The subtraction also reduces accuracy, by 26 to 49 points in Llama and by 8 to 57 points in Qwen. Part of this drop is not specific to the language. A random vector of the same norm already costs 8 to 25 points in Llama and 4 to 16 in Qwen, and for the French patch in Qwen the drop is the same as with a random vector. After manually reading the answers, we find that most of the errors come from two sources either the model replaces the question with a different one and answers it, or it asks the user for clarification without providing an answer. Llama tends to say that the question is written in a made-up language and translates it into a different question (37% of answers), while Qwen tends to say that the question contains typos and asks to rephrase it (31%). This kind of behavior emerges due to the fact that the inverse of a language is not well defined. After all, what does it mean to subtract French from a French text? Is it just keeping the content but discarding the form? The following are three very illustrative examples of this divergence.

> **User** ($-v_{\text{fr}}$): *Qui a rédigé le Manifeste du parti communiste?*[1]
>
> **Llama:** *I think I see what's going on here! It looks like you're trying to speak in Latin, but with a bit of a twist. The phrase you're using is a playful way of asking What is the meaning of the Communist Manifesto? The correct Latin phrase would be Quid est significatio manifesti communis? But I'm happy to help you with your question! The Communist Manifesto, written by Karl Marx and Friedrich Engels in 1848...*

The model responds in English and it correctly identifies the content/meaning. At first it haves trouble understanding

However, it recognizes the language as latin

but it doesn't identifies the question

The model doesn't understand the language of the question, it even says it is not widely known! However, it does understand the content and later correctly identifies it as french even though it doesn't perceive it as such.

> **User** ($-v_{\text{fr}}$): *Quel est le nombre de lunes que Mars possède?*[2]
>
> **Model:** *Creo que hay un pequeño error en tu pregunta. La respuesta correcta sería: ¿Cuál es el nombre de lunes que precede a Marte? La respuesta es: Mercurio.*[3]

Here the model reads the whole question as Spanish: *nombre* (French for *number*) becomes the Spanish *nombre* (*name*), *lunes* (French for *moons*) becomes the Spanish *lunes* (*Monday*), and the answer, *Mercurio*, follows the sequence of weekdays, whose Spanish names derive from the planets (*lunes*, *martes*, *miércoles*). See more example of this behaviors on the Annex.

Now the controls (table 6), we digest them in three parts. First, subtracting the patch of another language barely changes the language of the answer, Spanish and German questions do not exhibit any change (100% stays) and 90–96% of French ones are still answered in the language of the question. Second, random vectors with the same norm as the patch,

[1] "Who wrote the Communist Manifesto?"

[2] "How many moons does Mars have?"

[3] "I think there is a small error in your question. The correct question would be: What is the name of Monday that precedes Mars? The answer is: Mercury."

Table 5: Controls for the subtraction ($n = 50$ per row). Stays: percentage of answers that remain in the language of the question; answers with no identifiable language (e.g., a bare name such as *Marie Curie*) count as staying. Random: three Gaussian vectors rescaled to the norm of the patch of that language, one per seed. Question-3 and Header-3: position-bound patches trained on the same data (Section 3.1).

Question in	Subtracted	Stays (%)	Accuracy (%)
French	French patch	12	70
	Spanish patch	90	78
	German patch	96	76
	Random (3 seeds)	96–100	84–92
	Question-3	98	90
	Question-3, $\alpha = -2$	90	84
	Header-3	100	88
Spanish	Spanish patch	42	72
	French patch	100	82
	German patch	100	82
	Random (3 seeds)	100	84–88
German	German patch	60	82
	French patch	100	90
	Spanish patch	100	84
	Random (3 seeds)	100	86–90

generated with three different seeds, leave 96–100% of the answers in the language of the question. Third and last, the position-bound patches, which we test only on French questions, leave 98% (Question-3) and 100% (Header-3) of the answers in French; even doubling the magnitude of Question-3 ($\alpha = -2$) only reduces this to 90%, with accuracy dropping from 90% to 84%. In all cases, the effect is far smaller than that of subtracting the patch of the question's own language, which leaves only 12%, 40%, and 60% of French, Spanish, and German questions in their language.

We can safely conclude that the language patches behave as directions. Subtracting a patch from a question in its own language moves the answer out of that language, strongly for French and partially for Spanish and German, whereas subtracting the patch of another language affects the answer barely more than a random vector of the same norm. The position-bound patches, in contrast, cannot be inverted by subtraction.

Connecting with the fact that these patches do not change the tokens of the question (section 4.1) we sketch figure 1 which shows a visual representation of the above effect.

4.4 The patch behaves as a 'translation'

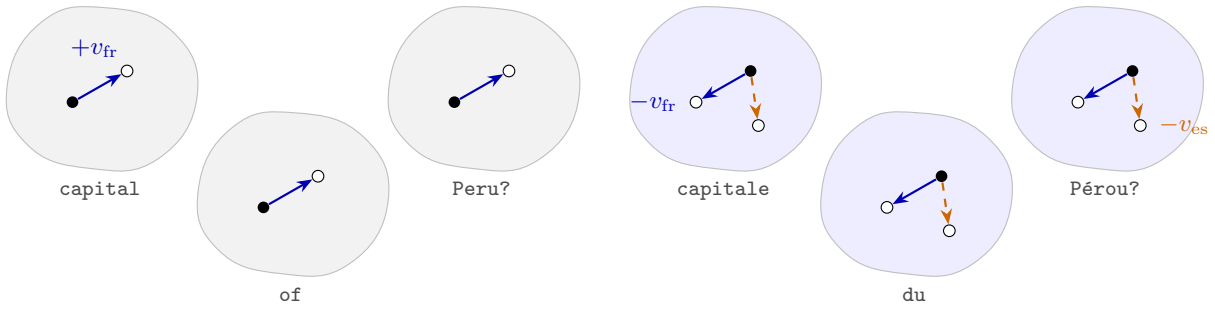
In the previous section, the errors of the model on subtracted patches gave a glimpse of the mechanism behind the language change. We now take a deep dive and ask if the patch is changing the input representation (translation) or is acting as an instruction. We now show the results of experiments detailed in Section 3.5.

4.4.1 Language identification

Table 6: Language identification on held-out English questions ($n = 50$ per row). The patch is applied only to the question, inside a message that asks which language it is written in (Section 3.5). Columns: percentage of answers that name French or English as the language of the question.

Question	Says French (%)	Says English (%)
English, no patch	0	100
English + v_{fr}	76	12
English + Question-3	44	28
English + Header-3	6	76

The rows do not sum to 100% (table 7) because the model sometimes names a third language, usually one associated with the topic of the question rather than the language it is written in (e.g., *Arabic* for the capital of Syria, *Norwegian* for the author of *The Scream*). This happens in 12% of the answers with the French patch, 28% with Question-3, and 18%



(a) Adding v_{fr} to an English question

"represented" as French

(b) Subtracting from a French question

$-v_{fr}$: "represented" outside French
$-v_{es}$: still "represented" as French

Figure 1: Schematic of the patch in the input embedding space (not to scale). Each shaded region is the neighborhood of one token of the question. (a) The same vector v_{fr} is added to every token of an English question: each token moves by the same displacement but stays within its own neighborhood, so the text of the question is unchanged, yet the question is "represented" as French. (b) Subtracting v_{fr} from the tokens of a French question moves them in the opposite direction, out of the French "representation", whereas subtracting the Spanish direction v_{es} moves them along a different axis and leaves it unchanged.

with Header-3, and never without a patch. Under Header-3, these answers are also written in French (e.g., *L'allemand*), and one of them does not name a language at all.

4.4.2 Subtraction against an instruction

We prepend `Answer this in French.` to the held-out English questions and subtract a patch from the question tokens only. If the patch encoded a decision about the output language, subtracting it should work against the instruction and if the patch is a 'translation' then it should have no effect on the answer because whatever the language is still the instruction is to answer in French. Table 8 shows that result, there is no effect on the answer, when subtracting v_{fr} it keeps a 100% rate.

Table 7: Subtraction against an instruction ($n = 50$ per row). The instruction `Answer this in French.` is prepended to the held-out English questions, and a patch is subtracted from the question tokens only. French: percentage of answers in French.

Condition	French (%)	Accuracy (%)
Instruction, no patch	100	96
Instruction − French patch	100	94
Instruction − random	100	96

4.4.3 Conflicting instruction

We now prepend `Answer this in English.` to a patched English question and compare with the same instruction applied to a question genuinely written in French (Table 8). Following the same logic as before, if the patch is an instruction then it should compete with the English one and we should have mixed results. If it is not then it should answer in English with no problem.

As a summary, the three tests point in the same direction. The model reports a question patched with v_{fr} as written in French, subtracting the patch does not counteract an explicit instruction, and an explicit instruction overrides it. The patch therefore behaves as a property of the input: It changes how the model reads the question, not what it is asked to do. Header-3 shows the opposite profile, the model still perceives the question as English, yet answers in French and even overrides an explicit instruction, as a compressed instruction would. Question-3 lies in between.

Table 8: Conflicting instruction ($n = 50$ per row): percentage of answers in English, without and with the instruction `Answer this in English.` prepended to the question. The patch is applied only to the question. Answers left unclassified by the language detector (short sentences) were labeled manually.

	English (%)	
Question	no instruction	with instruction
French, no patch	0	42
English + French patch	0	94
English + Question-3	2	98
English + Header-3	0	6

References